

Introduction à l'assembleur

CYT-VX8



I. La compilation : du langage de programmation au langage machine

Rappel : la compilation

- Langage compilé (C, C++, Rust ...) vs interprété (Python, Java ...).
- La compilation est la traduction complète du code source en code machine (binaire) avant son exécution. Une des étapes de la compilation est le passage par le **langage assembleur**.
- Les langages interprétés vont également utiliser l'assembleur puisqu'ils utilisent des fonctions déjà pré compilées en assembleur.
- Nous allons ici prendre en exemple le C comme langage classique compilé.



Rappel : GCC : phase de compilation

- Maintenant nous allons voir les différentes phases de l'exécutable **gcc** et les options associées :
- PREMIERE ETAPE :
gcc -E main.c -o main.i
- L'option **-E** de gcc (pré-traitement) permet de voir en sortie le programme C qui sera effectivement compilé
 - Le code non compilé entre les instructions **#ifndef** / **#ifdef** ne va plus apparaître
 - Toutes les macros ont été remplacées par leurs valeurs
 - L'ensemble des fichiers inclus par le fichier C est fusionné dans l'ordre d'inclusion : il n'y a donc plus qu'un seul fichier à compiler.

Rappel : GCC : phase de compilation

- Maintenant nous allons voir les différentes phases de l'exécutable **gcc** et les options associées :

- DEUXIEME ETAPE :

gcc -S main.i -o main.s

- L'option **-S** de gcc (compilation), permet de transformer le code C précédent en code **assembleur** (toujours lisible par l'humain...)

Rappel : GCC : phase de compilation

- Maintenant nous allons voir les différentes phases de l'exécutable **gcc** et les options associées :

- TROISIEME ETAPE :

gcc -c main.s -o main.o

- L'option **-c** de gcc (compilation) permet de **transformer le code assembleur en code machine**, c'est à dire en un code que seul le composant cible peut interpréter.

II. Assembleur et principes

L'assembleur : un langage au plus près du matériel

- Très souvent, une instruction en langage de programmation se fait en plusieurs étapes par la machine.

- Exemple :

$c = a + b ;$

Pour effectuer ce code le programme doit:

1. Récupérer la valeur de a en mémoire
2. et la mettre dans un registre du processeur
3. Idem avec b
4. Effectuer l'addition (dans l'ALU)
5. Charger le résultat là où est la variable c

- Ce découpage des actions à réaliser se fait lors de la compilation.

- Plus un langage est explicite sur les étapes d'instruction machine, plus il est **bas niveau**.

L'assembleur : un langage au plus près du matériel

- L'assembleur est un langage **très bas niveau**.
- En architecture RISC (reduced instruction set computer)

Une instruction assembleur ⇔ une instruction machine (en RISC)

- L'assembleur est apparu dans les années 50 pour remplacer la programmation binaire pure.
- Une instruction en assembleur est traduisible directement en langage machine aussi appelé **opcode**.
- Exemple :

ADD R3, R1, R2 (ajouter le contenu de R1 et de R2 et le stocker dans R3)

010101 00011 00001 00010

Qu'est ce que l'assembleur

- Chaque type de processeur a son jeu d'instruction et donc son propre assembleur dédié.
- => Il existe de nombreux langages assembleurs différents !
- Quelques exemples :

Processeur	Syntaxe	Famille d'assembleur
Intel/AMD	add eax, R2	x86
ARM (smartphones)	ADD R3, R2, R1	ARM
MIPS (embarqué)	add \$t3, \$t2, \$t1	MIPS
CYT-VX8	ADD R3, R1, R2	Simplifiée, proche de l'ARM

III. Concepts généraux de l'assembleur

Instructions classiques de l'assembleurs

- Nous utiliserons dans cette partie **l'assembleur ARM** car sa syntaxe est simple.
- En ARM les registres de l'ALU sont noté:
 - R₁
 - R₂
 - R₃
 - ...
 - R₁₅
- La syntaxe générale d'une instruction classique est :

[OPÉRATION] [registre(s) destination], [registre(s) source(s)]

Instructions classiques de l'assembleurs

- Nous utiliserons dans cette partie **l'assembleur ARM** car sa syntaxe est simple.
- Instructions arithmétiques :

ARM	Equivalent
ADD R3, R1, R2	$R3 = R1 + R2$
SUB R3, R1, R2	$R3 = R1 - R2$
MUL R3, R1, R2	$R3 = R1 * R2$

- Instructions logiques :

ARM	Equivalent
AND R3, R1, R2	$R3 = R1 \& R2$ (opération bit à bit)
ORR R3, R1, R2	$R3 = R1 R2$ (bit à bit)
CMP R2, R1	$R2 == R1$
Autre comparaison...	

Instructions classiques de l'assembleurs

- Nous utiliserons dans cette partie l'assembleur ARM car sa syntaxe est simple.
- Instructions de transferts :

ARM	Equivalent
MOV R1, R2	R1=R2
LDR R1, [R2]	Si R2 contient une adresse mémoire, son contenu est chargé dans R1
STR R1, [R2]	On stocke le contenu de R1 dans l'adresse mémoire donné par R2

- Les registres peuvent contenir des adresses !

Les modes d'adressages

- On peut charger ou stocker une valeur ou une adresse dans l'instruction, sans passer par un registre intermédiaire!
- L'**adressage** est la méthode utilisée par une instruction assembleur pour spécifier où se trouvent les données (registres ou mémoire) qu'elle doit manipuler.
- On a (liste non exhaustive) :

Type d'adressage	Exemple d'instruction	Explication
Registre	ADD R4, R6, R3	Utilise les registres pour les opérandes
Immédiat	MOV R1, #42	Charge une valeur directement dans le registre (42 ici)
Direct	LDR R1, [0x1000]	On donne l'adresse de la donnée à chargé des R1 (ici 0x1000)
Indirect	LDR R1, [R4]	R4 contient l'adresse dans laquelle charger la donnée

Exemple

$c = a + b ;$

LDR R1, [a] ; Charge la valeur de 'a' dans R1

LDR R2, [b] ; Charge la valeur de 'b' dans R2

ADD R1, R1, R2 ; $R1 = a + b$

LDR R2, [c] ; Charge la valeur de 'c' dans R2

MUL R2, R1, R2 ; $R2 = (a + b) * c$

STR R2, [c] ; Stocke le résultat dans 'c' (écrase l'ancienne valeur de c)

➤ Rmq: [a] est un label qui représente l'adresse mémoire où a est stockée.

A l'exécution, l'instruction LDR R1, [a] devient LDR R1, 0x1001

Opcode et stockage des instructions en mémoire

- Chaque instruction = un opcode binaire
 - Une instruction est traduite en opcode binaire
 - Cet opcode vient en entrée du processeur qui peut la décoder et l'exécuter
- Les instructions binaires sont stockées dans la RAM
 - **A la suite, dans l'ordre à laquelle elles doivent être exécutées !** (pour les instructions séquentielles).
 - Chaque instruction peut avoir une taille différente.
- Exemple

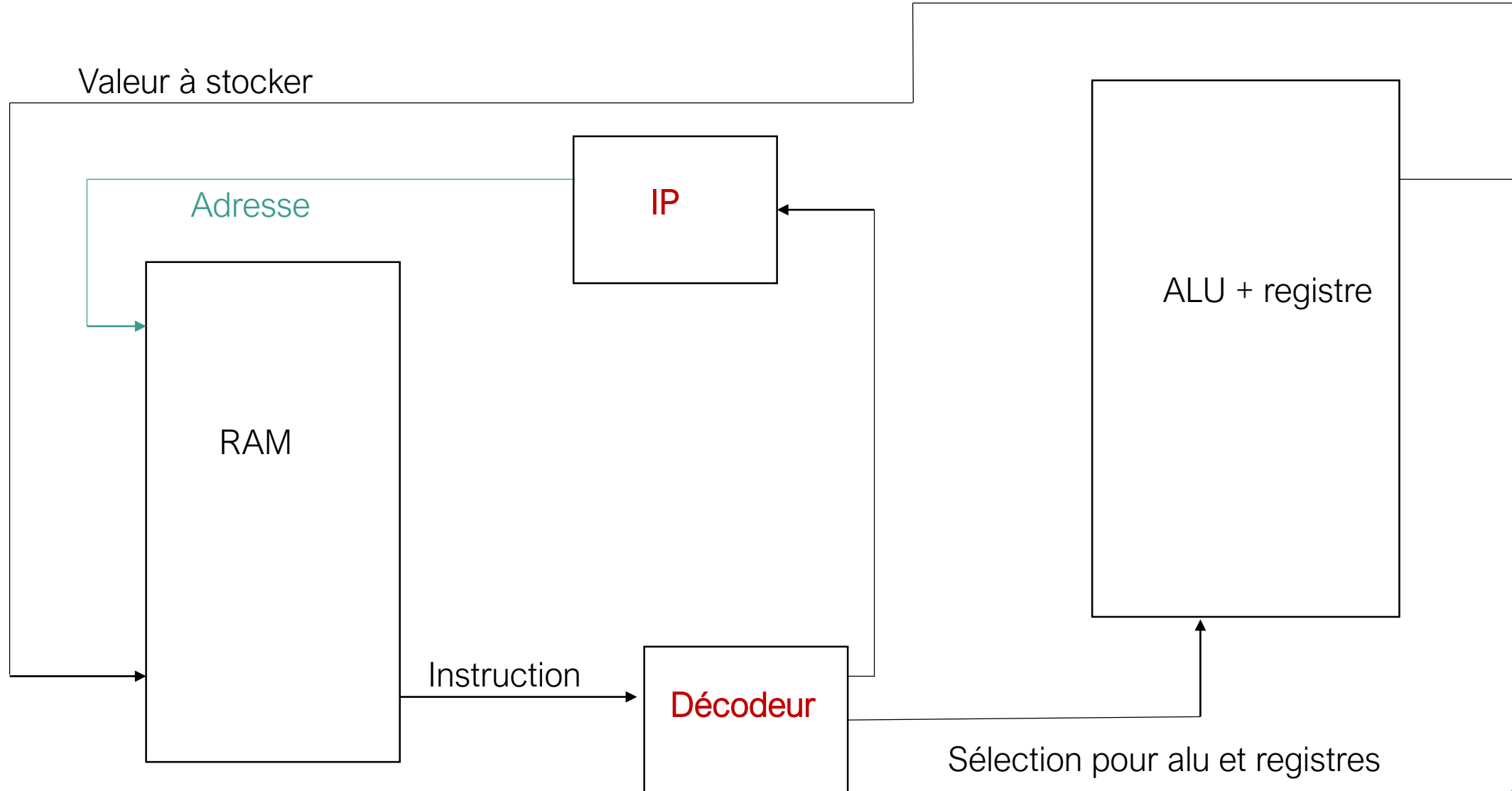
Adresse	Contenu (opcode)	Instruction	taille
0x100	1000 0011 0001 0010	ADD R3, R1, R2	2 octets
0x102	1100 0001 0001 1000	MOV R1, #8	2 octets
0x104	1010 0001 0001 0000 0000 0000	LDR R1, [0x1000]	3 octets
0x107	1110 0010 0001	STR R2, [R1]	2 octets

L'instruction Pointeur

- Afin de pouvoir la récupérer, **le processeur a besoin de connaître l'adresse de la prochaine instruction à exécuter.**
- Cette information est stockée dans l'**Instruction Pointeur (IP)** qui est stockée et calculée par le processeur. (aussi appelé **PC pour Program Counter**).
- L'IP:
 - ✓ Pointe vers l'adresse de la prochaine exécution
 - ✓ Est automatiquement incrémenté après chaque instruction (en fonction de sa taille).
 - ✓ Peut être modifié autrement pour certaines instructions
- Question : Quelle catégorie d'instructions fait que l'instruction suivante exécutée n'est pas celle qui la suit directement en mémoire ?

L'instruction Pointeur

- Un schéma très grossier



Les flags

- Pour les instructions conditionnelles ou les boucles, la prochaine instruction est conditionnée par un booléen, souvent résultat d'une comparaison.
- Les Flags sont des bits spéciaux (1 bit = 1 flag) stockés dans un registre d'état (appelé CPRS en ARM et tout simplement FLAG pour nous)
- Il permet de prendre des décisions
- Il peuvent être modifiés automatiquement par l'ALU à chaque fois qu'une opération va être effectuée
- On a par exemple :
 - V (overflow): signale un overflow sur les nombres signés
 - C (carry): signal un overflow sur les nombres non signés
 - Z (zero): signal que deux nombre d'une comparaison sont égaux
 - Rmq : il existe plus de flags plus compliqués mais ici nous simplifions.
- Pour simplifier beaucoup, **nous dirons ici que Z se met à 1 pour n'importe quelle comparaison donnant le résultat VRAI.**

Le flag Z

➤ Exemple :

```
MOV R1, #5  
MOV R2, #5  
CMP R1, R2 ; Compare R1 et R2 → Z = 1
```

```
MOV R1, #5  
MOV R2, #8  
CMP R1, R2 ; Compare R1 et R2 → Z = 0
```

- En fonction de la valeur de Z, le processeur va aller chercher une instruction plutôt qu'un autre.

Le saut conditionnel

- Après une comparaison, on peut « sauter » une partie du code si le flag Z=1 ou Z=0.
- Instructions :

ARM	Equivalent
BEQ LABEL	Saute sur Z=1
BNE LABEL	Saute si Z=0

- Exemple :

```
MOV R1, #10
MOV R2, #10
CMP R1, R2 ; Compare R1 et R2 → Z = 1
BEQ equal ; Saute à "equal" car Z = 1
MOV R3, #0 ; Cette ligne est ignorée
JMP end ; fin du saut
equal:  MOV R3, #1 ; R3 = 1 (car R1 == R2)end:
```

Le saut conditionnel

- Après une comparaison, on peut « sauter » une partie du code si le flag Z=1 ou Z=0.
- Instructions :

ARM	Equivalent
BEQ LABEL	Saute sur Z=1
BNE LABEL	Saute si Z=0

- Exemple :

```
MOV R1, #10
MOV R2, #10
CMP R1, R2 ; Compare R1 et R2 → Z = 1
BEQ equal ; Saute à "equal" car Z = 1
MOV R3, #0 ; Cette ligne est ignorée
JMP end ; fin du saut si
equal: MOV R3, #1 ; R3 = 1 (car R1 == R2)end:
```



```
If (10 != 10){
    // mettre une variable à 0
}

// mettre une variable à 1
```

- Les labels seront remplacés par les adresses des instructions lors de l'exécution/ passage en opcode.

Le saut conditionnel

➤ Même mécanisme pour les boucles :

loop:

ADD R1, R1, R2

ADD R2, R2, #1

CMP R2, #6

BNE loop ; Saute EN ARRIÈRE** vers "loop" si $R2 \neq 6$

end:

Stack et Stack pointer

- Comme leur code, les instructions d'une fonction seront à part dans la mémoire.
- Il faut donc pouvoir
 - A l'appel de la fonction : le programme « saute » vers la fonction
 - Au retour le programme reprend où il s'était arrêté.
 - Gérer les différents retours en cas d'appel successif
 - Passer des arguments
- Une **pile** (**stack**) située dans la RAM va stocker à chaque appel de fonction:
 - L'adresse d'instruction de retour (où revenir après la fonction)
 - L'état de registres avant l'appel
 - (Passer des arguments (si trop nombreux pour les registres))
- Un registre du processeur appelé **stack pointeur** (pointeur de pile) indique l'adresse du dessus de la pile.

Stack et Stack pointer

ARM	Equivalent
PUSH R1	Empiler R1 dans la pile
POP R1	Dépiler la pile dans R1

➤ Exemple d'utilisation

--- Début du programme ---

MOV R1, #10 ; R1 = 10 (valeur à sauvegarder)

PUSH R1 ; Sauvegarde R1 dans la pile (SP décrémenté, valeur 10 stockée)

--- On peut maintenant utiliser R1 pour autre chose ---

MOV R1, #20 ; R1 = 20 (on écrase l'ancienne valeur)

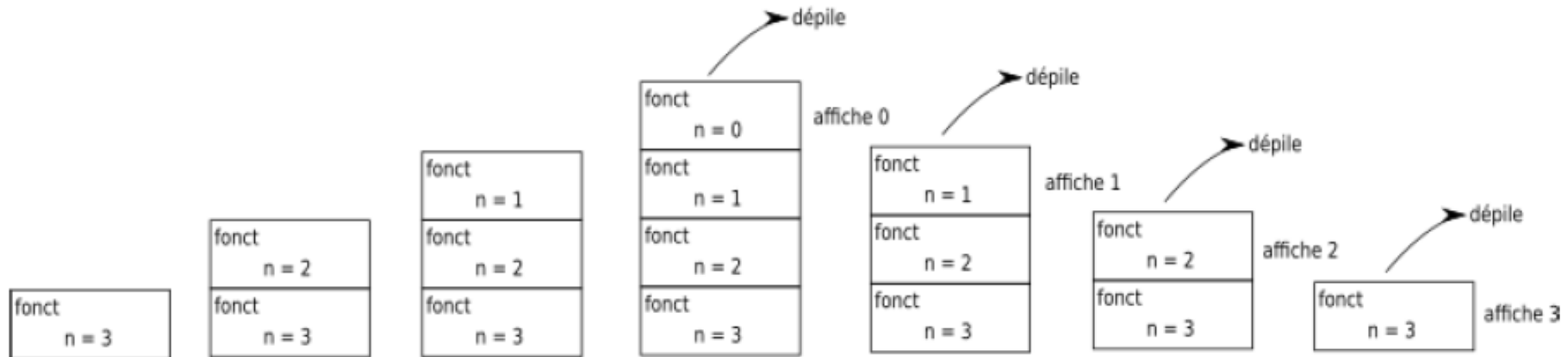
ADD R1, R1, #5 ; R1 = 20 + 5 = 25;

--- Restaurer R1 à sa valeur initiale ---

POP R1 ; R1 = 10 (valeur restaurée depuis la pile, SP incrémenté);

--- Fin du programme : R1 = 10 (comme au début) ---

Illustration pour les fonctions



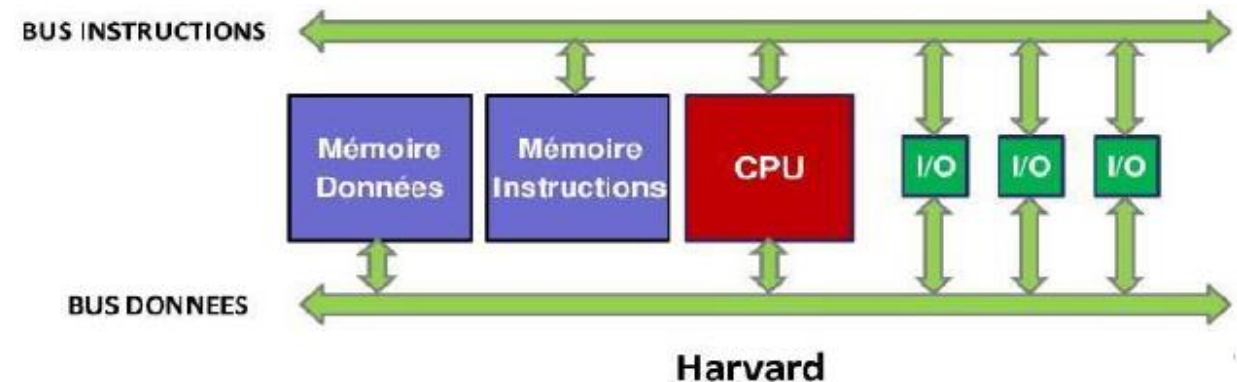
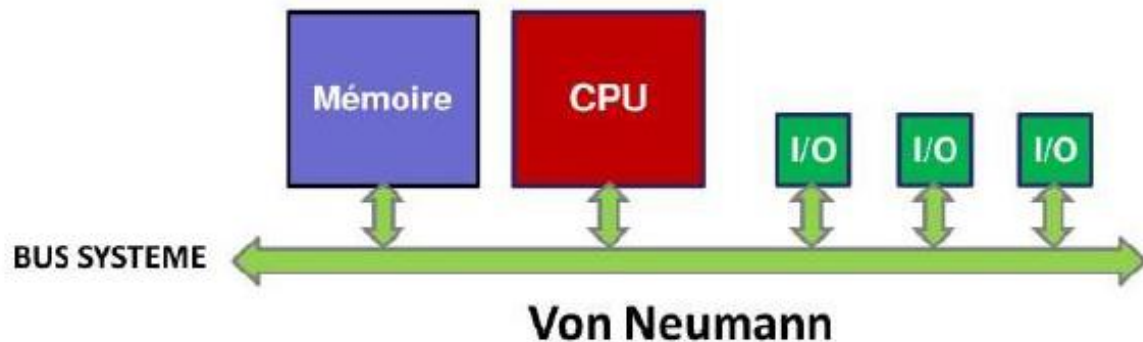
L'assembleur CYT VX8

Pour notre processeur le CYT VX8

- Beaucoup de simplification
 - ✓ Pas de Stack et de SP
 - ✓ Un seul flag : le résultat d'une comparaison précédente
 - ✓ Les instructions seront toutes sur **24 bits**.
 - ✓ Pas de fonctions
- ✓ Ajout d'une instruction NOP : une instruction qui fait juste passer un cycle d'horloge sans rien changer (pour des besoins que nous verrons plus tard).
- ✓ Deux RAM : une pour les données et une pour les instructions

Rappel

- Sur la majorité des ordinateurs, les instructions et les données sont stockées dans la même mémoire vive. C'est **l'architecture de Von Newman** (contrairement à **l'architecture d'Harvard**).
 - Une seule mémoire
 - 1 bus d'adresse, 1 bus de données
 - Avantage : gestion simplifiée de la mémoire
 - Inconvénient : goulot d'étranglement pour l'accès à la mémoire.
- Nous travaillerons donc sur de l'Harvard !



Exercice d'illustration

- Executons en assembleur CYT VX8 le code suivant :

```
unsigned char k = 0;
for(unsigned char i=1;i<=4;i++){
    k += i;
}
```



```
NOP
# i, const and k
CNST, 01, R1
CNST, 04, R2
CNST, 00, R3
# Loop start (check)
CMP_LE, R1, R2
JNE, 09
ADD, R3, R1, R3
# Loop end (incr + back)
INC, R1
JMP, 04
# store k
STOR, R3, 00
```

Exercice

- Executons en assembleur CYT VX8 le code suivant :

```
NOP  
# i, const and k  
CNST, 01, R1  
CNST, 04, R2  
CNST, 00, R3  
# Loop start (check)  
CMP_LE, R1, R2  
JNE, 09  
ADD, R3, R1, R3  
# Loop end (incr + back)  
INC, R1  
JMP, 04  
# store k  
STOR, R3, 00
```

NOP : ne rien faire

Exercice

- Executons en assembleur CYT VX8 le code suivant :

```
NOP
# i, const and k
CNST, 01, R1
CNST, 04, R2
CNST, 00, R3
# Loop start (check)
CMP_LE, R1, R2
JNE, 09
ADD, R3, R1, R3
# Loop end (incr + back)
INC, R1
JMP, 04
# store k
STOR, R3, 00
```

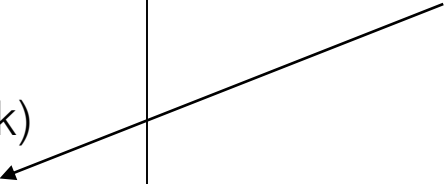
← CNST constante registre : mettre la constante dans le registre

Exercice

- Executons en assembleur CYT VX8 le code suivant :

```
NOP
# i, const and k
CNST, 01, R1
CNST, 04, R2
CNST, 00, R3
# Loop start (check)
CMP_LE, R1, R2
JNE, 09
ADD, R3, R1, R3
# Loop end (incr + back)
INC, R1
JMP, 04
# store k
STOR, R3, 00
```

CMP_LE : compare least or equal.
Le résultat va dans le flag Z

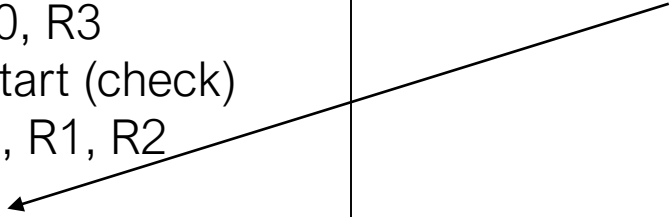


Exercice

- Executons en assembleur CYT VX8 le code suivant :

```
NOP
# i, const and k
CNST, 01, R1
CNST, 04, R2
CNST, 00, R3
# Loop start (check)
CMP_LE, R1, R2
JNE, 09
ADD, R3, R1, R3
# Loop end (incr + back)
INC, R1
JMP, 04
# store k
STOR, R3, 00
```

JNE Adresse : si le flag est à 0 aller à l'instruction de l'adresse

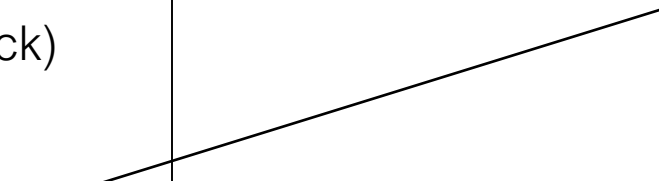


Exercice

- Executons en assembleur CYT VX8 le code suivant :

```
NOP
# i, const and k
CNST, 01, R1
CNST, 04, R2
CNST, 00, R3
# Loop start (check)
CMP_LE, R1, R2
JNE, 09
ADD, R3, R1, R3
# Loop end (incr + back)
INC, R1
JMP, 04
# store k
STOR, R3, 00
```

ADD R3, R1, R3 : $R3 = R1 + R3$

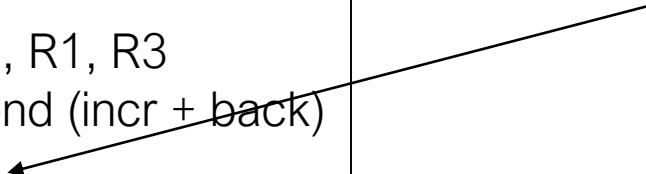


Exercice

- Executons en assembleur CYT VX8 le code suivant :

```
NOP
# i, const and k
CNST, 01, R1
CNST, 04, R2
CNST, 00, R3
# Loop start (check)
CMP_LE, R1, R2
JNE, 09
ADD, R3, R1, R3
# Loop end (incr + back)
INC, R1
JMP, 04
# store k
STOR, R3, 00
```

INC = incrementer



Exercice

- Executons en assembleur CYT VX8 le code suivant :

```
NOP
# i, const and k
CNST, 01, R1
CNST, 04, R2
CNST, 00, R3
# Loop start (check)
CMP_LE, R1, R2
JNE, 09
ADD, R3, R1, R3
# Loop end (incr + back)
INC, R1
JMP, 04
# store k
STOR, R3, 00
```

JMP adresse : retourner à l'adresse

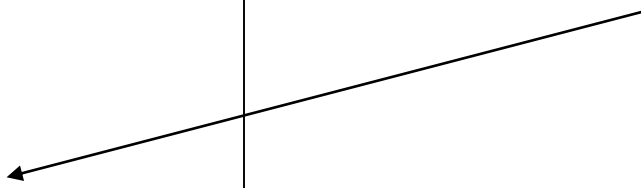


Exercice

- Executons en assembleur CYT VX8 le code suivant :

```
NOP
# i, const and k
CNST, 01, R1
CNST, 04, R2
CNST, 00, R3
# Loop start (check)
CMP_LE, R1, R2
JNE, 09
ADD, R3, R1, R3
# Loop end (incr + back)
INC, R1
JMP, 04
# store k
STOR, R3, 00
```

STOR registre adresse : store la valeur de R3 à l'adresse indiquée



Exercice

- Notre RAM d'instruction stocke 3 octets par adresse soit une instruction.
- Ici, la RAM instruction sera ;

Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

```
# i, const and k
CNST, 01, R1
CNST, 04, R2
CNST, 00, R3
# Loop start (check)
CMP_LE, R1, R2
JNE, 09
ADD, R3, R1, R3
# Loop end (incr + back)
INC, R1
JMP, 04
# store k
STOR, R3, 00
```

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
0	0	0	0	0

Rmq : la première instruction sera toujours à l'adresse 0. IP vaut donc 0.
IP vaut toujours l'adresse de la prochaine instructions à réaliser.

Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
0	0	0	0	1

NOP : On ne change pas les valeurs des registres.
Par défaut IP est incrémenté.



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
1	0	0	0	2



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
1	4	0	0	3



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
1	4	0	0	4



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
1	4	0	1	5



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
1	4	0	1	?

JNE : si le flag est à 0 on va à l'adresse indiquée



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
1	4	0	1	6

JNE : si le flag est à 0 on va à l'adresse indiquée



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
1	4	1	1	7



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
2	4	1	1	8



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
2	4	1	1	4



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
2	4	1	1	5



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
2	4	1	1	6



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
2	4	3	1	7



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
3	4	3	1	8



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
3	4	3	1	4



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
3	4	3	1	5



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
3	4	3	1	6



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
3	4	6	1	7



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
4	4	6	1	8



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
4	4	6	1	4



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
4	4	6	1	5



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
4	4	6	1	6



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
4	4	10	1	7



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
5	4	10	1	8



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
5	4	10	1	4



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
5	4	10	0	5



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
5	4	10	0	9



Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

Exercice

- Execution pas à pas du code vu du processeur:

R1	R2	R3	Z	IP
5	4	10	0	10

Sauvegarde de R3 dans la RAM de données

Adresse	Instruction contenue
0x00	NOP
0x01	CNST, 01, R1
0x02	CNST, 04, R2
0x03	CNST, 00, R3
0x04	CMP_LE, R1, R2
0x05	JNE, 09
0x06	ADD, R3, R1, R3
0x07	INC, R1
0x08	JMP, 04
0x09	STOR, R3, 00

